PG AND HOSTEL MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

THIVYAA P

(240701570)

in partial fulfilment of the End Semester Examination for the course

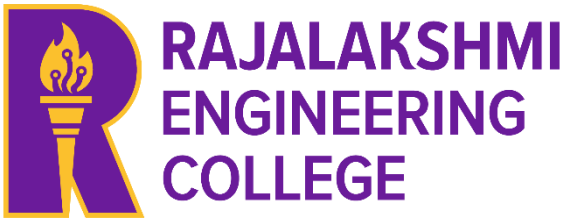
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "PG and Hostel Management System" is the bonafide work of Thivyaa P (240701570), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms. Aswana Lal
COURSE FACULTY
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Thivyaa P (240701570), hereby declare that the project report titled “PG and Hostel Management System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 27 October 2026

SIGNATURE OF THE STUDENT
Thivyaa P
240701570

ACKNOWLEDGEMENT

I am grateful to our Principal, Dr S N Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J Manoranjini , Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Ms. Aswana Lal Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

Thivyaa P

ABSTRACT

Paying-guest (PG) accommodations and small hostels usually record residents, room allocations, monthly rent, complaints and visitors in paper registers or loosely maintained spreadsheets. This makes it easy to allocate a room that is already full or under maintenance, to lose track of unpaid rent and to overlook maintenance complaints, while the owner has no single view of occupancy and dues.

This project developed the PG and Hostel Management System on Zoho Creator using the platform's drag-and-drop builder, without AI-assisted generation. The application contains six linked forms (Resident, Room, Allocation, Payment, Complaint and Visitor Log), lookup fields that relate them, and a formula field that computes the balance due on every payment. Deluge scripts validate allocations and payments by cancelling the submission with an alert when the check-out date precedes the check-in date, when a room is not available or is already full, or when the amount paid is not positive. Form workflows mark a room as Occupied once it reaches capacity and email the administrator when a complaint is logged, a scheduled workflow prepares a weekly rent-due reminder, and an approval process governs resident registration. Eight list reports and a dashboard page with three charts and an embedded Pending Payments report give the owner an overview, while Warden and Resident permission profiles and a role hierarchy control access.

The application was tested in the live Development environment with clearly fake data. Duplicate emails, rooms under maintenance, full rooms, reversed dates and zero payments were all blocked as intended, and the formula field, reports and charts produced the expected values. Two items could not be confirmed: delivery of the notification emails to the administrator's inbox and the first run of the weekly schedule, which starts on 2 November 2026. The work shows that a complete, rule-enforcing management application can be built on Zoho Creator without writing a conventional program, with Deluge needed only for a few validations and notifications.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Hostel Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	PG and Hostel Management System
Application Domain	Hospitality / Residential Accommodation Management (PG and Hostel)
Student Name & Register No.	Thivyaa P – 240701570
Zoho Creator Edition	Free Trial (Zoho Creator, India data centre)
Application Name (Link Name)	pg-and-hostel-management-system
Published / Live URL	https://creatorapp.zoho.in/ (Development environment; application link name above)
Demo Credentials (per role)	Admin: owner account (credentials to be shown at the demonstration) | Warden, Resident: roles defined, no users invited
Number of Forms / Subforms	6 / 0
Number of Reports (by type)	List: 8 (6 auto-generated form reports + Pending Payments + Room Occupancy) | Kanban: 0 | Calendar: 0 | Pivot/Chart: 0 chart reports (3 charts on the Dashboard page) | Others: 0
Number of Pages / Dashboards	1 (Dashboard page with 3 charts and an embedded report)
Number of Workflows	Form: 4 | Schedule: 1 | Approval: 1 | Blueprint: 0 | Batch: 0
Deluge Functions / Custom Actions	0 custom functions / 5 Deluge scripts (DS-01 to DS-05)
Integrations	Zoho Creator built-in email (sendmail) to the application administrator
Roles & Profiles Defined	Administrator (CEO, default top role), Warden, Resident
Environments Used	Development only (Stage and Production not used)
Date of Production Deployment	Not deployed to Production (application used in the Development environment)

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development 5
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 7
2.6 Proposed System 7
REQUIREMENT ANALYSIS 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 11
SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 Data Model 14
4.3 Form Design 15
4.4 Process and Workflow Design 18
4.5 User Interface and Navigation Design 19
4.6 Access Control Design 19
IMPLEMENTATION 20
5.1 Application Setup 20
5.2 Forms and Data Capture 20

5.3 Reports 22
5.4 Pages and Dashboards 23
5.5 Deluge Scripting 24
5.6 Workflows and Automation 26
5.7 Integrations 28
5.8 User Management and Role-Based Access 28
5.9 AI and Analytics Features 29
5.10 Pro-Code Extension 29
5.11 Mobile and Portal Access 29
TESTING AND DEBUGGING 30
6.1 Testing Strategy 30
6.2 Test Cases 30
6.3 Debugging Deluge Scripts 32
6.4 Defect Log 33
6.5 User Acceptance Testing 33
DEPLOYMENT, SECURITY AND GOVERNANCE 34
7.1 Environments and Deployment Strategy 34
7.2 Performance Optimisation 34
7.3 Security Features 35
7.4 Governance and Administration 35
RESULTS AND DISCUSSION 37
8.1 Application Output 37
8.2 Achievement of Objectives 38
8.3 Comparison with the Existing System 39
8.4 Limitations of the Low-Code Approach 39
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 39
CONCLUSION AND FUTURE ENHANCEMENTS 41
9.1 Conclusion 41
9.2 Future Enhancements 41
9.3 Learning Outcomes and Career Relevance 41
REFERENCES 43
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 44
APPENDIX B – COURSE OUTCOME MAPPING 47
APPENDIX C – COMPLETE DELUGE SOURCE CODE 48
APPENDIX D – USER MANUAL 50
D.1 Accessing the Application 50
D.2 Instructions for Administrator 50

D.3 Instructions for Warden 50
D.4 Instructions for Resident 51
APPENDIX E – EVALUATION RUBRIC 52
APPENDIX F – VIVA-VOCE RECORD SHEET 54

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 10
Table 3.4: Hardware and Software Requirements 10
Table 3.5: Use Case Description – Allocate a Room to a Resident 11
Table 4.1: Entities and Relationships 14
Table 4.2: Field Specification – Resident Form 15
Table 4.3: Field Specification – Room Form 16
Table 4.4: Field Specification – Allocation Form 16
Table 4.5: Field Specification – Payment Form 16
Table 4.6: Field Specification – Complaint Form 17
Table 4.7: Field Specification – Visitor Log Form 17
Table 4.8: Role–Permission Matrix 19
Table 5.1: Reports Implemented 22
Table 5.2: Deluge Scripts Implemented 24
Table 5.3: Workflows Implemented 27
Table 5.4: Integrations Implemented 28
Table 6.1: Test Cases 30
Table 6.2: Defect Log 33
Table 6.3: User Acceptance Feedback 33
Table 7.1: Deployment Checklist 34
Table 7.2: Performance Optimisation Results 35
Table 7.3: Security Controls Implemented 35
Table 8.1: Mapping of Objectives to Outcomes 38
Table 8.2: Existing System versus Proposed Application 39
Table A.1: Feature Coverage Checklist 44
Table B.1: Mapping of Course Outcomes to Report Chapters 47
Table E.1: Evaluation Rubric for the Individual Project 52
Table E.2: Performance Level Descriptors 53
Table F.1: Viva-Voce Questions and Responses 54
Table F.2: Marks Summary 54

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 4
Figure 3.1: Use Case Diagram of the Application 11
Figure 4.1: System Architecture of the Application 13
Figure 4.2: Entity–Relationship Diagram 14
Figure 4.3: Workflow / Process Flow for Room Allocation 18
Figure 4.4: Navigation Structure of the Application and Layout of the Dashboard Page 19
Figure 5.1: Application Home and Navigation Sections 20
Figure 5.2: Allocation Form in the Form Builder 21
Figure 5.3: Allocation Form in Live Mode 22
Figure 5.4: Pending Payments Report 23
Figure 5.5: Dashboard Page 24
Figure 5.6: Workflow Configuration 27
Figure 5.7: Roles and Permission Profiles Configuration 29
Figure 6.1: Validation Alert Shown on Submission of the Allocation Form 32
Figure 8.1: Scenario Step 1 – Allocation to a Room Under Maintenance Is Blocked 37
Figure 8.2: Scenario Step 2 – Allocation Saved and Room Becomes Occupied 37
Figure 8.3: Scenario Step 3 – Payment with Balance Due and Pending Payments Report 38

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DS	Deluge Script
ER	Entity–Relationship
FR	Functional Requirement
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
PG	Paying Guest
RBAC	Role-Based Access Control
REST	Representational State Transfer
TC	Test Case
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

A paying-guest (PG) accommodation or hostel rents beds or rooms to students and working professionals on a monthly basis. Even a small establishment must keep track of who lives in which room, how many beds are still free, which rooms are under maintenance, how much rent each resident has paid, which complaints are open and who has visited the residents. These tasks are normally shared between the owner, who looks at occupancy and dues, and a warden or caretaker, who handles day-to-day registration, rent collection and complaints.

In most small PGs this information is kept in a paper admission register, a rent ledger, a gate register for visitors and a notebook or phone messages for complaints. The owner prepares a monthly summary by hand. The process works while the number of residents is small, but each register is separate, so no one can see the whole picture at once. This project builds a single low-code application on Zoho Creator for a hypothetical PG with a few floors, one owner (administrator), one warden and the residents.

1.2 Problem Statement

The manual, register-based process lets errors enter at the point of data entry and gives the owner no real-time view of occupancy or dues. A room can be allocated twice, a resident can be registered with a duplicate email, an incorrect payment can be recorded, and complaints can wait unnoticed. The pain points are:

- Room allocation is not checked against room status and capacity, so a full or under-maintenance room can be allocated.
- Rent dues are calculated by hand from a ledger, so the balance for a resident is slow to find and often wrong.
- Complaints are not notified to the owner automatically and have no status tracking.
- Resident, room, payment and visitor details are scattered across separate registers, with duplicate or missing entries.
- There are no role-based restrictions, and the owner has no dashboard or report for occupancy and pending payments.

1.3 Objectives

- To design a normalised data model and forms for residents, rooms, allocations, payments, complaints and visitors.
- To automate allocation and payment validation, room status updates, complaint alerts and a weekly rent reminder using Deluge scripting and workflows.
- To provide role-based reports and a dashboard of occupancy, payments and complaints for the owner and the warden.
- To use Zoho Creator's built-in email (sendmail) integration to notify the administrator of complaints and pending rent.
- To record the deployment, security and governance status of the application honestly, including what was not done.

1.4 Scope of the Project

In scope: six forms (Resident, Room, Allocation, Payment, Complaint, Visitor Log); validations on allocation and payment; automatic room status update; complaint email alert; weekly rent-due reminder; an approval process for resident registration; list reports including Pending Payments and Room Occupancy; one dashboard page with three charts and an embedded report; and permission profiles and roles for Admin, Warden and Resident. The application is used on the web, in the Development environment of a Zoho Creator free trial, with clearly fake test data only.

Out of scope: online payment collection, billing and invoicing, meal management, biometric or gate-pass integration, a resident self-service portal, the mobile app, external REST API integrations, and publishing to Stage or Production environments.

1.5 Justification for a Low-Code Approach

Low-code platforms let applications be assembled mainly through visual configuration, with code written only where the logic is specific to the business [1]. For a PG owner this is attractive for four reasons. First, the data model, forms and reports needed here are standard, so the builder produces them in far less time than a hand-coded web application. Second, Zoho Creator hosts the application and provides the database, security and web access, so there is no server to set up or maintain [2]. Third, the few rules that are specific to a hostel, such as blocking a full room or a negative payment, are short enough to be written in Deluge, Zoho's scripting language [3]. Fourth, an

owner or a non-programmer warden can later change a field, report or workflow without a development team.

This fits the definition of a Low-Code Application Platform (LCAP): visual modelling for most of the work, with a small amount of scripting for custom logic.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

At present the PG owner and warden follow a paper-based process. A new resident fills in a printed admission form, and the warden copies the details into an admission register. The warden decides the room from memory or from a room notebook and marks it as taken. Rent is collected in cash or by UPI, and the amount is written in a rent ledger. Complaints are told to the warden in person or sent as phone messages, and visitors are noted in a gate register. At the end of the month the owner prepares a summary of occupancy and dues by hand from these records.

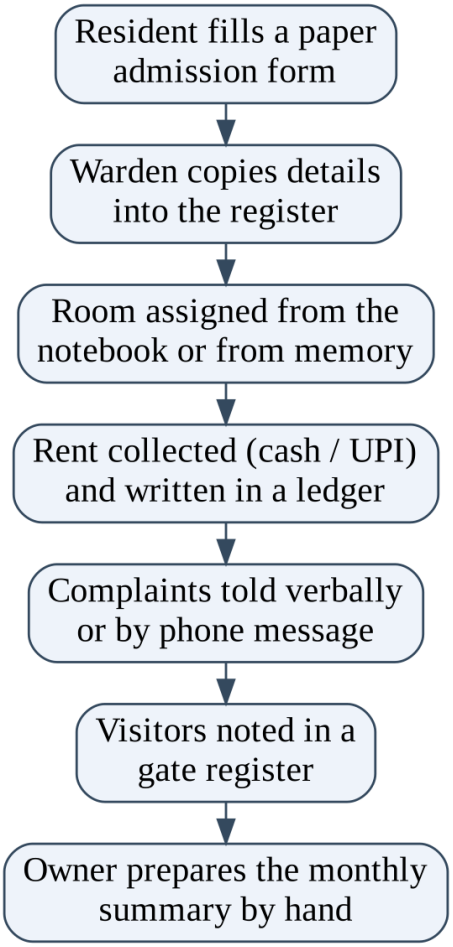


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- No validation: a full room, a room under maintenance, an earlier check-out date or a zero payment is not rejected.

- Duplicate and incomplete resident records, because there is no unique-email rule or mandatory field.
- No automatic balance calculation; the due amount for each resident is computed manually.
- Complaints are not tracked by status and the owner is not alerted.
- No role-based access: anyone who can open the register can read or change any record.
- Reports (occupancy, pending rent) take manual effort and become outdated quickly.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Estimated 6–8 weeks for design, database, screens, validations, reports, login and hosting	Estimated days rather than weeks: forms, reports and the dashboard are configured visually
Skills required	Web front end, back end, database and deployment skills	Data modelling and basic Deluge scripting; a citizen developer can maintain it
Hosting & infrastructure	Server, database and domain to be arranged and maintained	Hosted by Zoho Creator; no server set-up
Mobile application	Separate mobile or responsive build needed	Zoho Creator mobile app is available (not used in this project)
Security & access control	Login, roles and data protection coded by hand	Built-in roles, permission profiles, unique and mandatory field rules
Maintenance & change	Code change, testing and redeployment for each change	Edit a field, report or workflow in the builder; changes are live in Development immediately
Customisation limits	No platform limits, full freedom	Limited to platform features; some report and permission settings are restricted (see Section 8.4)
Cost	Developer time plus hosting cost	Platform subscription (a free trial was used for this project)

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. C. Bock and U. Frank, 2021 [1]	Low-Code Platform	Describes low-code platforms as environments for building applications mainly through visual models, with limited scripting.	Defines the approach used for the hostel application.
2	Zoho Corporation [2]	Zoho Creator Help Documentation	Documents forms, reports, pages, workflows, roles and hosting in Zoho Creator.	Reference for every builder feature used in Chapter 5.
3	Zoho Corporation [3]	Deluge Script Reference Guide	Explains Deluge syntax, fetch, collections, alert, cancel submit and sendmail.	Basis for scripts DS-01 to DS-05.
4	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, 2020 [4]	Supporting the understanding and comparison of low-code development platforms	Compares low-code platforms by their features and shows that they differ in modelling, logic and deployment support.	Supports the choice of Zoho Creator for a small data-driven application.
5	Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, 2021 [5]	Characteristics and challenges of low-code development: the practitioners' perspective	Reports practitioners' views: faster delivery is a main benefit, while platform limits and customisation are common challenges.	Matches the benefits and limits found in this project (Section 8.4).
6	M. Waszkowski, 2019 [6]	Low-code platform for automating business processes in manufacturing	Shows a low-code platform automating a business process with forms, workflows and rules.	Shows the same pattern of forms plus automated workflow used here.
7	J. Cabot, 2020 [7]	Positioning of the low-code movement within the field of model-driven engineering	Relates low-code to model-driven engineering and discusses its strengths and open issues.	Background for the spectrum discussion in Section 8.5.

2.5 Emerging Trends in Low-Code Development

Several trends in low-code development influenced this project. AI-assisted application building, such as Zoho's Zia and the AI application builder, can generate forms and scripts from a prompt. For this project the application was built manually in the builder and no Zia or AI field was used, so that every design decision was made and understood directly. Citizen development, in which non-programmers build and maintain applications, is the reason the design uses simple forms, lookups and short Deluge scripts [7]. Hyper-automation is reflected in the chained automation of room status updates, complaint alerts and the weekly reminder, and the composable approach is reflected in the separate forms joined by lookups, so that a new form such as a Maintenance Request can be added later without changing the others [6]. A recent textbook on low-code practice also stresses that low-code and professional development complement each other [8].

2.6 Proposed System

The proposed system is the PG and Hostel Management System on Zoho Creator. It addresses each limitation in Section 2.2 as follows: Deluge validations reject a full or unavailable room, a reversed date range and a non-positive payment; the unique Email rule and mandatory fields prevent duplicate and incomplete resident records; the Balance Due formula field computes dues automatically; the Complaint form has a status field and an email alert to the administrator; permission profiles and roles restrict who can open which form; and reports and a dashboard give the owner occupancy, pending payments and complaint status without manual summaries.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	The PG owner. Holds the top role (CEO, the default top role in Zoho Creator) and owns the application account.	Manage roles and settings, approve resident registrations, receive complaint and rent alerts, view all data and the dashboard.
Warden	Caretaker who runs the PG day to day. Mid-level role under the Administrator.	Register residents, manage rooms, allocate rooms, record payments and visitors, view reports and the dashboard.
Resident	A person staying in the PG. Lowest role.	Raise complaints and view the dashboard.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow a resident to be registered with name, phone, email, gender, ID type and guardian details; email shall be unique.	Admin, Warden	High
FR-02	The system shall route each new resident registration to the administrator for approval.	Admin	High
FR-03	The system shall maintain a room master with room number, floor, type, capacity, monthly rent and status.	Admin, Warden	High
FR-04	The system shall allocate a resident to a room with check-in date, optional check-out date and deposit.	Warden	High
FR-05	The system shall reject an allocation whose check-out date is earlier than the check-in date.	Warden	High
FR-06	The system shall reject an allocation if the room status is not Available or the room is already full.	Warden	High
FR-07	The system shall automatically set a room to Occupied when its active allocations reach its capacity.	System	High
FR-08	The system shall record rent payments and reject an amount paid of 0 or less.	Warden	High
FR-09	The system shall automatically compute Balance Due as the room’s monthly rent minus the amount paid.	System	High

FR ID	Requirement	Role(s)	Priority
FR-10	The system shall record complaints with category, description and status, and email the administrator on submission.	Resident, Admin	Medium
FR-11	The system shall send the administrator a weekly rent-due reminder listing the count and total balance of pending payments.	System	Medium
FR-12	The system shall log visitors against a resident with in and out times.	Warden	Low
FR-13	The system shall provide list reports for all forms, a Pending Payments report and a Room Occupancy report.	Admin, Warden	Medium
FR-14	The system shall display a dashboard with charts of rooms by type, payments by status and complaints by status.	Admin, Warden, Resident	Medium
FR-15	The system shall restrict forms and pages according to the Warden and Resident permission profiles.	Admin	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with up to 500 records shall load within 5 seconds (platform hosted).
NFR-02	Security	Users shall access only the forms and pages permitted by their permission profile.
NFR-03	Usability	The application shall be usable in a standard web browser without installation.
NFR-04	Availability	The application shall be hosted on the Zoho Creator cloud, which manages uptime and backups.
NFR-05	Maintainability	Forms and fields shall use consistent, meaningful link names so that scripts remain readable.
NFR-06	Auditability	Record changes shall be traceable through the Zoho Creator audit trail (not reviewed in this project).

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator Free Trial (India data centre: creator.zoho.in; live app creatorapp.zoho.in)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome (latest version)
API testing tool	Not used (no REST API integration)
Other Zoho services	Zoho Creator built-in email (sendmail) only
Mobile	Not used
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

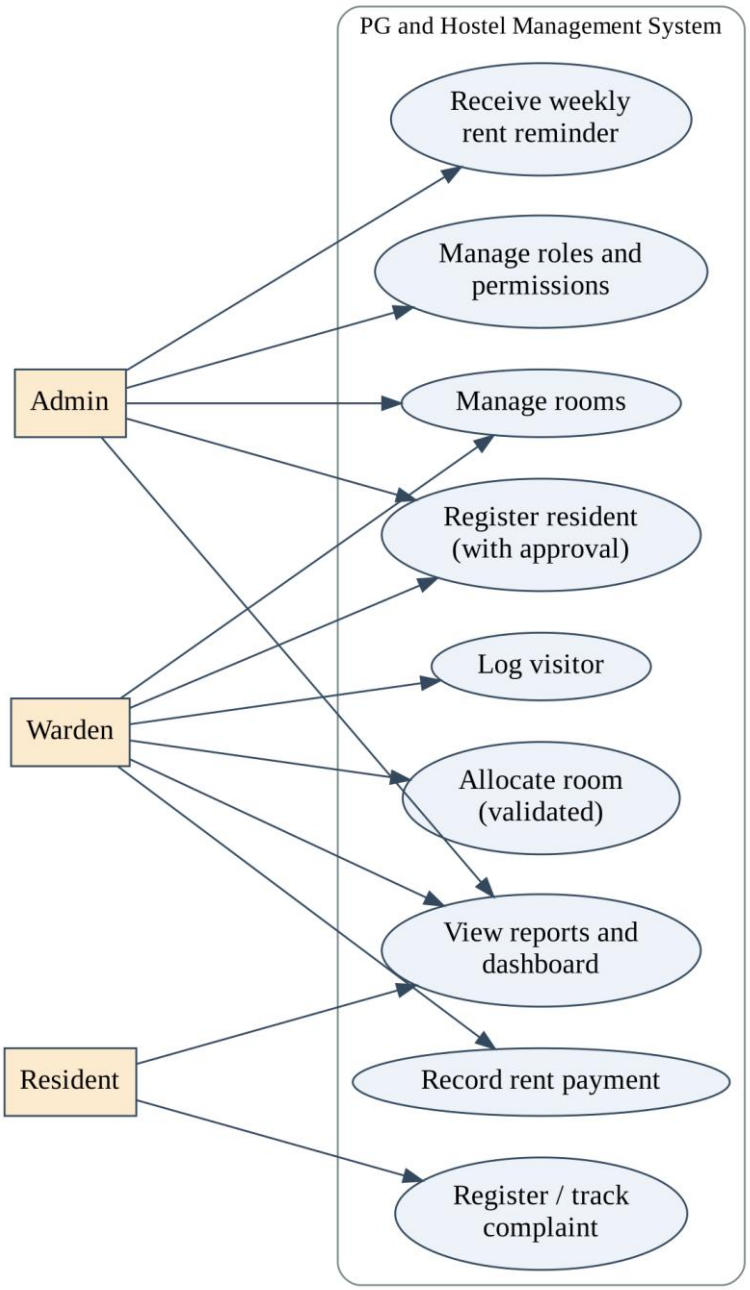


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Allocate a Room to a Resident

Use Case ID	UC-01
Use Case Name	Allocate a room to a resident
Actor(s)	Warden
Pre-condition	The resident and the room already exist; the warden has access to the Allocation form.

Main Flow	1. Warden opens the Allocation form. 2. Warden selects the Resident and Room and enters the check-in date and deposit. 3. Warden submits the form. 4. The system validates the dates, the room status and the room capacity (DS-01). 5. The record is saved and the room is set to Occupied if it is now full (DS-03).
Alternate Flow	If the check-out date is before the check-in date, or the room is not Available, or the room is full, the system shows an alert and cancels the submission; the warden corrects the entry or chooses another room.
Post-condition	The allocation is stored and the room status reflects its occupancy.
Related FR	FR-04, FR-05, FR-06, FR-07

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

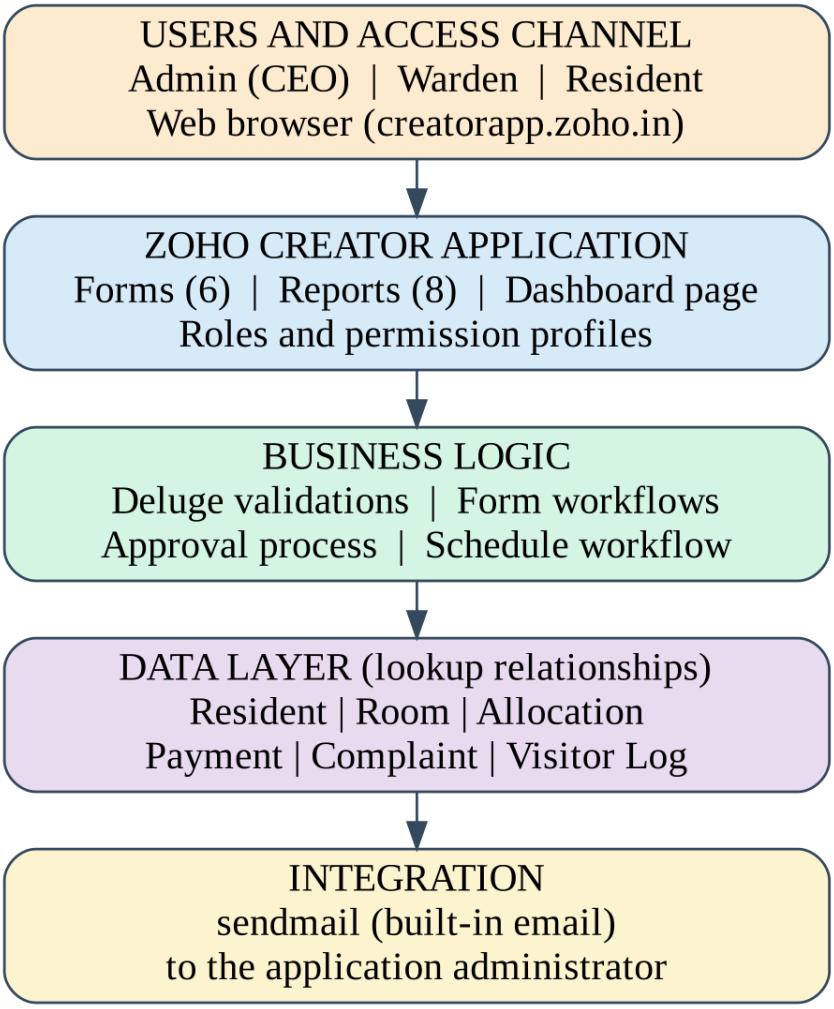


Figure 4.1: System Architecture of the Application

Figure 4.1 shows the layered architecture. At the top, three kinds of user (Administrator, Warden and Resident) reach the application through a web browser. The Zoho Creator application layer contains six forms, eight list reports, the Dashboard page and the roles and permission profiles. Business logic is placed in a separate layer made up of Deluge validation scripts, form workflows, the Resident Registration Approval process and the weekly schedule workflow. All records are stored in the Zoho Creator data layer as six related forms, joined by lookup fields. The only integration is the built-in sendmail task, which sends email to the application administrator.

4.2 Data Model

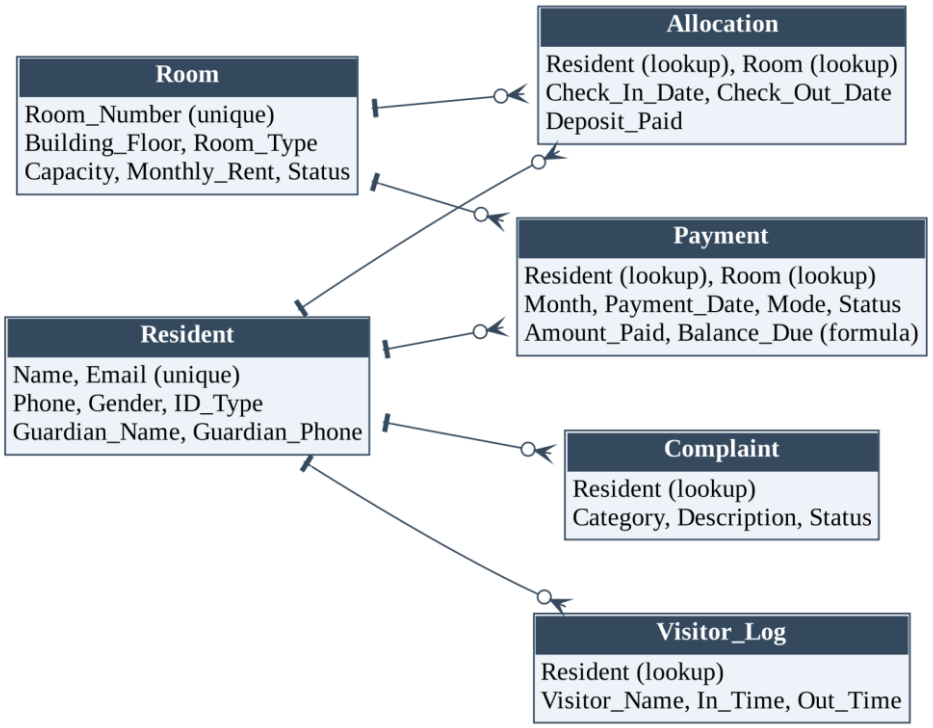


Figure 4.2: Entity–Relationship Diagram

Figure 4.2 is the entity–relationship diagram. Resident and Room are the two master entities. Allocation links one resident to one room for a period, and Payment records rent paid by a resident for a room. Complaint and Visitor Log each belong to one resident. In Zoho Creator these one-to-many relationships are implemented as lookup fields; no subform was needed because each child record (allocation, payment, complaint, visit) is created on its own form and is not entered as rows inside a parent record. Table 4.1 lists the entities.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Resident	Name, Email (unique), Phone, Gender, ID_Type, Guardian_Name, Guardian_Phone	Allocation, Payment, Complaint, Visitor_Log	One-to-Many	Lookup in each child form
Room	Room_Number (unique), Building_Floor, Room_Type, Capacity, Monthly_Rent, Status	Allocation, Payment	One-to-Many	Lookup in Allocation and Payment

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Allocation	Resident, Room, Check_In_Date, Check_Out_Date, Deposit_Paid	Resident, Room	Many-to-One	Lookup fields
Payment	Resident, Room, Month, Payment_Date, Mode, Status, Amount_Paid, Balance_Due	Resident, Room	Many-to-One	Lookup fields; Balance_Due is a Formula field
Complaint	Resident, Category, Description, Status	Resident	Many-to-One	Lookup field
Visitor_Log	Resident, Visitor_Name, In_Time, Out_Time	Resident	Many-to-One	Lookup field

4.3 Form Design

Table 4.2 to Table 4.7 give the field specification of each form. Field labels and link names follow one convention (Title_Case with underscores) so that Deluge scripts can refer to them reliably.

Table 4.2: Field Specification – Resident Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Name	Name	Name	No	—	Resident's full name
Email	Email	Email	Yes	No duplicate values	Unique key for a resident
Phone	Phone	Phone	Yes	—	
Gender	Gender	Radio	No	—	Male / Female / Other
ID Type	ID_Type	Drop Down	No	—	Aadhaar / Passport / Driving Licence
Guardian Name	Guardian_Name	Single Line	No	—	
Guardian Phone	Guardian_Phone	Phone	No	—	

Table 4.3: Field Specification – Room Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Room Number	Room_Number	Single Line	Yes	No duplicate values	Unique key for a room
Building Floor	Building_Floor	Single Line	No	—	
Room Type	Room_Type	Drop Down	No	—	Single / Double / Triple
Capacity	Capacity	Number	Yes	—	Number of beds; used by DS-01 and DS-03
Monthly Rent	Monthly_Rent	Decimal	Yes	—	Used by the Balance_Due formula
Status	Status	Drop Down	No	—	Available / Occupied / Under Maintenance

Table 4.4: Field Specification – Allocation Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Resident	Resident	Lookup	No	—	Lookup to Resident
Room	Room	Lookup	No	—	Lookup to Room (shows Room Number)
Check In Date	Check_In_Date	Date	Yes	—	
Check Out Date	Check_Out_Date	Date	No	Not earlier than Check In (DS-01)	Empty while the resident is staying
Deposit Paid	Deposit_Paid	Decimal	No	—	

Table 4.5: Field Specification – Payment Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Resident	Resident	Lookup	No	—	Lookup to Resident
Room	Room	Lookup	No	—	Lookup to Room
Month	Month	Single Line	No	—	For example Nov-2026
Payment Date	Payment_Date	Date	No	—	

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Mode	Mode	Drop Down	No	—	Cash / UPI / Card
Status	Status	Drop Down	No	—	Paid / Pending / Partial
Amount Paid	Amount_Paid	Decimal	Yes	Greater than 0 (DS-02)	
Balance Due	Balance_Due	Formula	—	Room.Monthly_Rent − Amount_Paid	System calculated

Table 4.6: Field Specification – Complaint Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Resident	Resident	Lookup	No	—	Lookup to Resident
Category	Category	Drop Down	No	—	Electrical / Plumbing / Cleaning
Description	Description	Multi Line	No	—	
Status	Status	Drop Down	No	—	Open / In Progress / Resolved

Table 4.7: Field Specification – Visitor Log Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Resident	Resident	Lookup	No	—	Lookup to Resident
Visitor Name	Visitor_Name	Single Line	Yes	—	
In Time	In_Time	Date-Time	No	—	
Out Time	Out_Time	Date-Time	No	—	

4.4 Process and Workflow Design

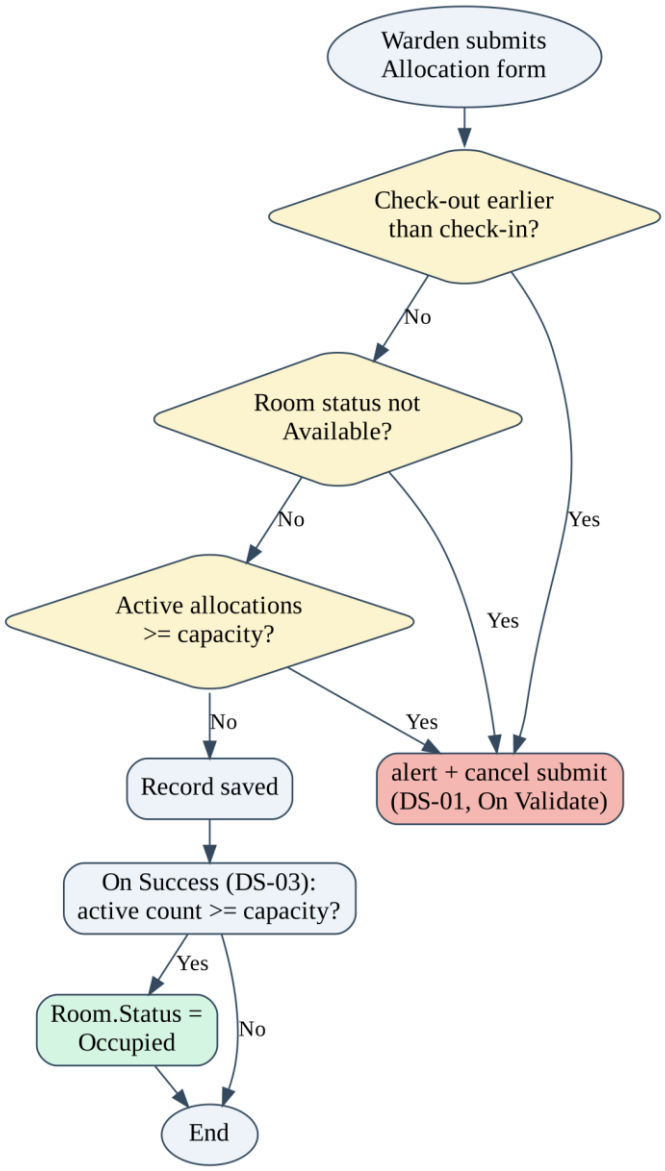


Figure 4.3: Workflow / Process Flow for Room Allocation

Figure 4.3 shows the process flow for the most important automated process, room allocation. DS-01 runs On Validate and cancels the submission with an alert if the dates, the room status or the room capacity fail the check. When the record is saved, DS-03 runs On Success, counts the active allocations of the room and sets the room status to Occupied if the count has reached the capacity. Payment validation (DS-02), the complaint email alert (DS-04), the weekly reminder (DS-05) and the resident approval follow the same pattern and are described in Section 5.6.

4.5 User Interface and Navigation Design

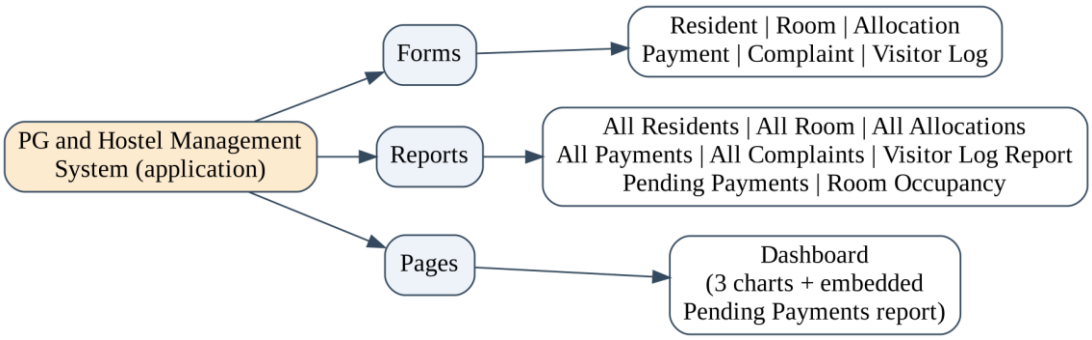


Figure 4.4: Navigation Structure of the Application and Layout of the Dashboard Page

Figure 4.4 shows the navigation structure of the application: a Forms section with the six forms, a Reports section with the eight list reports and one Dashboard page that shows three charts and the Pending Payments report. The dashboard is the single page used by all three roles.

4.6 Access Control Design

Table 4.8 defines the permission matrix that Section 5.8 implements. Creator's View, Edit and Delete options open per-report pickers, so the Warden and Resident profiles were limited to form-level Access plus the Dashboard. A = Add, E = Edit, V = View, D = Delete, Access = form access granted at profile level, — = no access.

Table 4.8: Role–Permission Matrix

Component	Administrator	Warden	Resident
Form: Resident	A E V D	Access	—
Form: Room	A E V D	Access	—
Form: Allocation	A E V D	Access	—
Form: Payment	A E V D	Access	—
Form: Complaint	A E V D	Access	Access
Form: Visitor Log	A E V D	Access	—
Page: Dashboard	V	Access	Access

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created from scratch in my Zoho Creator account on the India data centre (creator.zoho.in), using the "Create application" option and the title "PG and Hostel Management System". The AI-assisted "Create with AI" option was deliberately not used; every form, field, workflow and report was configured manually in the builder. The application has link name pg-and-hostel-management-system and the default theme. Its menu has three sections: Forms (the six forms), Reports (the eight list reports) and the Dashboard page.

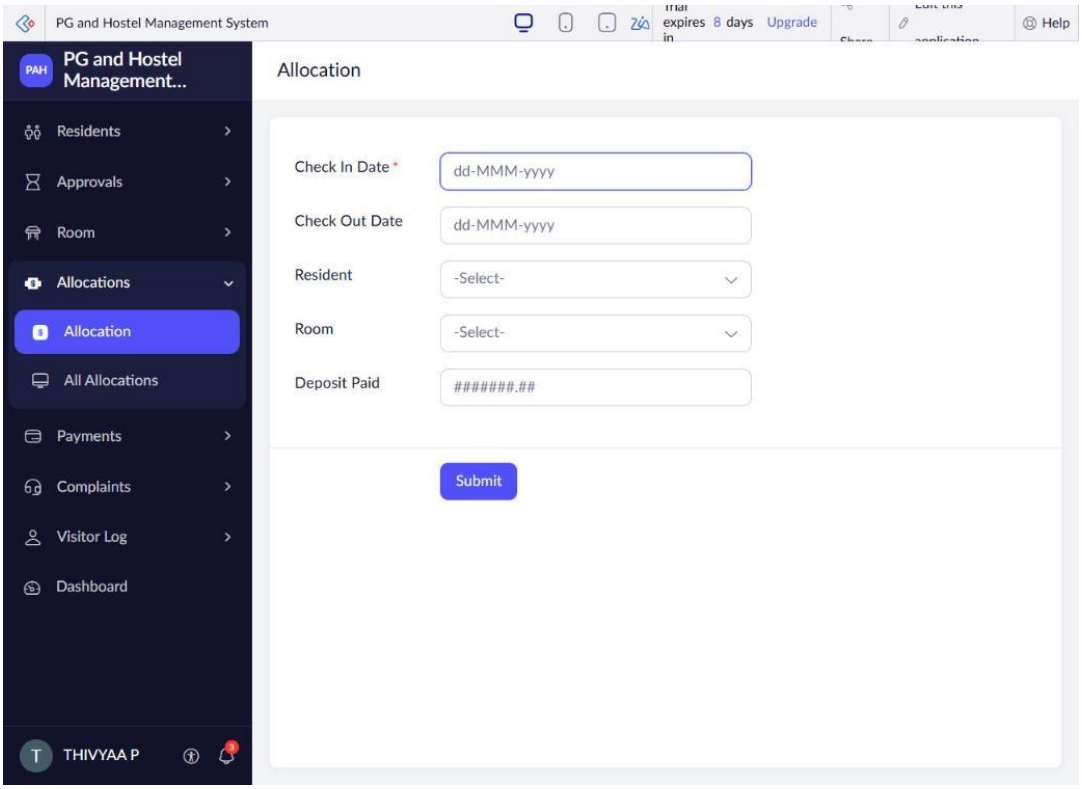


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Resident: Holds personal details. Email is mandatory with the No-duplicate-values property, and Phone is mandatory. Gender is a Radio field and ID Type is a Drop Down. An approval process (Section 5.6) is attached to this form.

Room: Room Number is mandatory and unique, Capacity and Monthly Rent are mandatory, and Room Type and Status are Drop Down fields. Status is changed to Occupied by workflow DS-03.

Allocation: Resident and Room are Lookup fields; Check In Date is mandatory. DS-01 validates the entry and DS-03 updates the room.

Payment: Resident and Room are Lookup fields; Amount Paid is mandatory. Balance Due is a Formula field that subtracts the amount paid from the room's monthly rent. DS-02 rejects an amount of 0 or less.

Complaint: Resident lookup, a Category drop down (Electrical, Plumbing, Cleaning), a Multi Line Description and a Status drop down (Open, In Progress, Resolved). Submission triggers DS-04.

Visitor Log: Resident lookup, a mandatory Visitor Name, and In Time and Out Time Date-Time fields.

The forms use twelve distinct field types: Name, Email, Phone, Single Line, Multi Line, Number, Decimal, Radio, Drop Down, Date, Date-Time, Lookup and Formula. Subforms and layout rules were not used (Section 4.2).

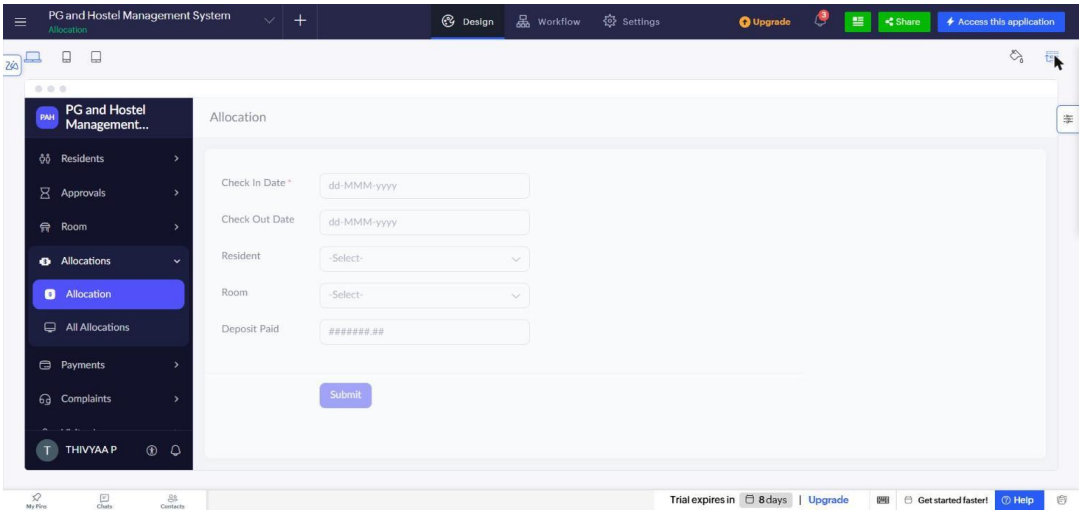


Figure 5.2: Allocation Form in the Form Builder

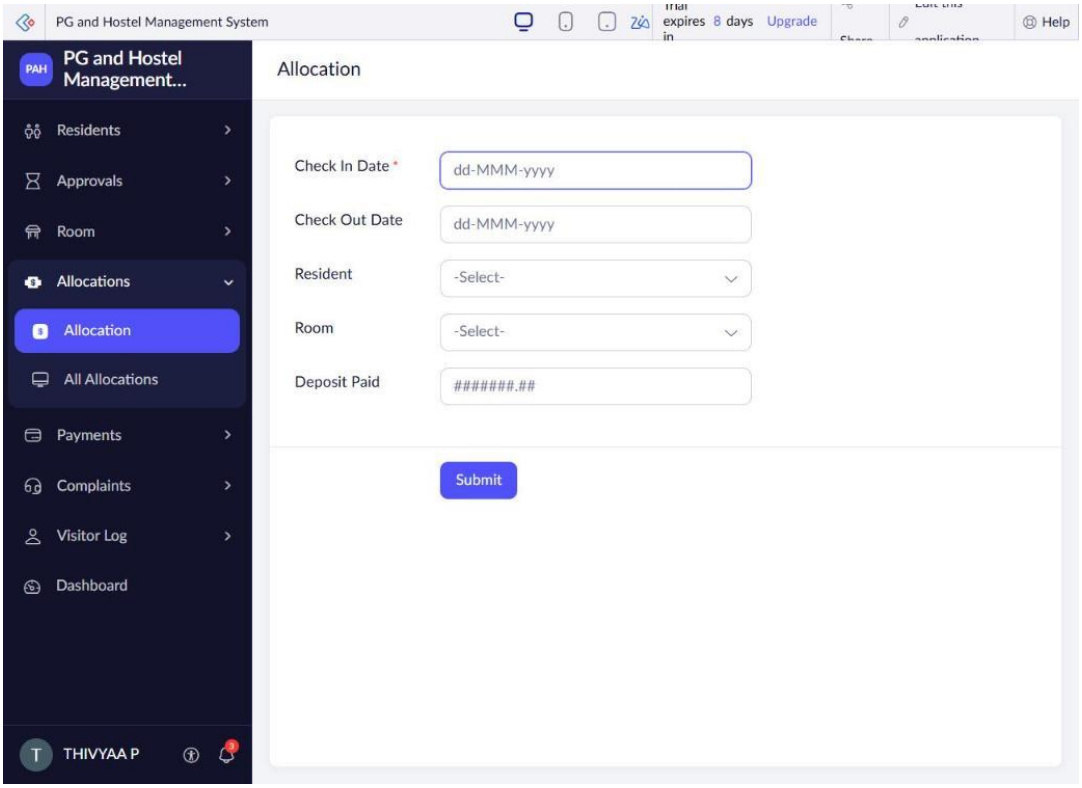


Figure 5.3: Allocation Form in Live Mode

5.3 Reports

Table 5.1 lists the eight list reports. Six were auto-generated when each form was saved. Pending Payments was created on the Payment form with the filter Status = Pending and sorted by Balance Due in descending order. Room Occupancy is a list report on Room that shows each room's status and capacity. Known gap: the Balance Due column is not offered in the column chooser of Pending Payments, so it is not shown in that report (it is visible on the Payment form). Kanban, Calendar, Timeline and Map reports were not built because the data has no pipeline, schedule or location view that needs them.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Residents	List	Resident	None (all records)	None	Admin, Warden
All Room	List	Room	None (all records)	None	Admin, Warden
All Allocations	List	Allocation	None (all records)	None	Admin, Warden

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Payments	List	Payment	None (all records)	None	Admin, Warden
All Complaints	List	Complaint	None (all records)	None	Admin, Warden, Resident
Visitor Log Report	List	Visitor Log	None (all records)	None	Admin, Warden
Pending Payments	List	Payment	Filter: Status = Pending; sorted by Balance Due, descending	None	Admin, Warden
Room Occupancy	List	Room	Columns: room number, floor, type, rent, status, capacity	None	Admin, Warden

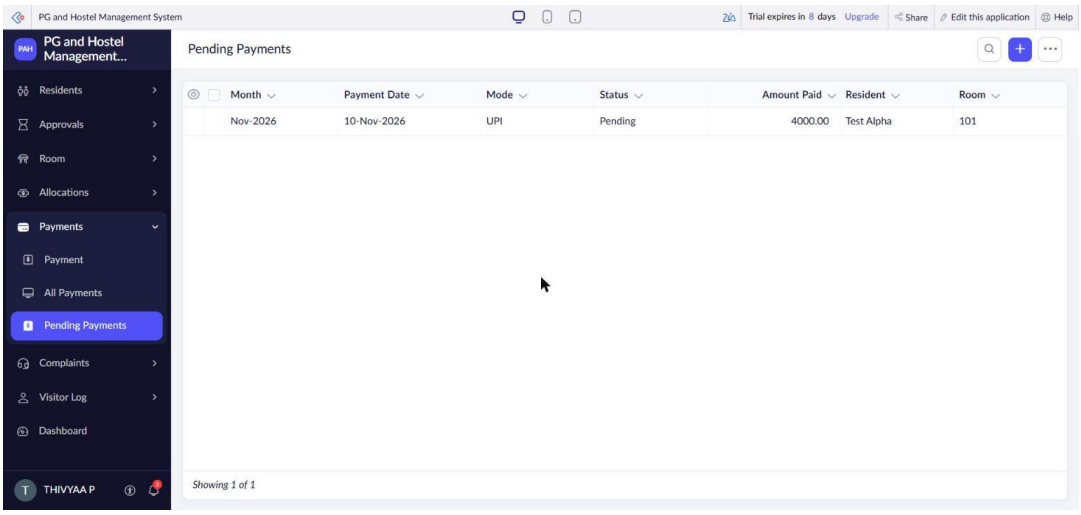


Figure 5.4: Pending Payments Report

5.4 Pages and Dashboards

The Dashboard page was built in the UI Builder. It has three column charts: Occupancy by Room Type (count of rooms by Room Type), Payments by Status (count by Status) and Complaints by Status (total records by Status). Below the charts, the Pending Payments list report is embedded so that the administrator sees dues next to the occupancy chart. The page is available to the Warden and Resident profiles as well as the Administrator.

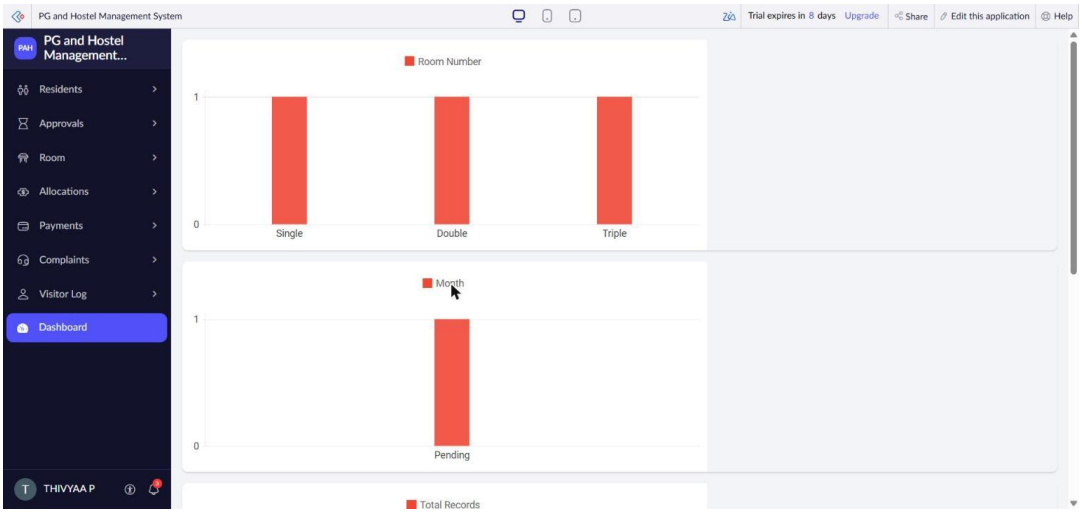


Figure 5.5: Dashboard Page

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Allocation → On Validate	Reject a reversed date range, a room that is not Available or a room that is full	if-else, fetch criteria, count(), alert, cancel submit
DS-02	Payment → On Validate	Reject an amount paid of 0 or less	if, alert, cancel submit
DS-03	Allocation → On Success	Set the room to Occupied when active allocations reach capacity	fetch, count(), update record
DS-04	Complaint → On Success	Email the administrator about a new complaint	sendmail, string concatenation
DS-05	Schedule: Weekly Rent Due Reminder	Email the administrator the count and total balance of pending payments	fetch criteria, for each, ifnull, sendmail

Code 5.1: Validate Allocation (DS-01) – Allocation → On Validate (Add)

```
// DS-01 | Allocation → On Validate (Add) | Block the submission if
the dates, room status or capacity are invalid
if(input.Check_Out_Date != null && input.Check_Out_Date <
input.Check_In_Date)
{
    alert "Check-out date cannot be earlier than the check-in date.";
    cancel submit;
}
roomRec = Room[ID == input.Room];
if(roomRec.Status != "Available")
{
    alert "This room cannot be allocated. Room status is: " +
roomRec.Status;
    cancel submit;
}
currentCount = Allocation[Room == input.Room && Check_Out_Date ==
null].count();
if(currentCount >= roomRec.Capacity)
{
    alert "This room is already full. Capacity: " + roomRec.Capacity;
    cancel submit;
}
```

Code 5.2: Validate Payment (DS-02) – Payment → On Validate (Add)

```
// DS-02 | Payment → On Validate (Add) | Block a payment whose amount
is 0 or less
if(input.Amount_Paid <= 0)
{
    alert "Amount paid must be greater than 0.";
    cancel submit;
}
```

Code 5.3: Update Room Status (DS-03) – Allocation → On Success (Add)

```
// DS-03 | Allocation → On Success (Add) | Set the room to Occupied
when it has reached capacity
roomRec = Room[ID == input.Room];
activeCount = Allocation[Room == input.Room && Check_Out_Date ==
null].count();
if(activeCount >= roomRec.Capacity)
{
    roomRec.Status = "Occupied";
}
```

Code 5.4: Complaint Email Alert (DS-04) – Complaint → On Success (Add)

```
// DS-04 | Complaint → On Success (Add) | Notify the application
administrator of a new complaint
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"New hostel complaint: " + input.Category
    message :"A new complaint was registered. Category: " +
input.Category + ". Description: " + input.Description + ". Status: "
+ input.Status
]
```

Code 5.5: Weekly Rent Due Reminder (DS-05) – Schedule workflow: weekly from 02-Nov-2026 09:00

```
// DS-05 | Schedule workflow: weekly from 02-Nov-2026 09:00 | Email
the administrator the count and total balance of pending payments
pendingList = Payment[Status == "Pending"];
cnt = pendingList.count();
total = 0;
for each p in pendingList
{
    total = total + ifnull(p.Balance_Due, 0);
}
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"Weekly rent due reminder"
    message :"Pending payments: " + cnt + ". Total balance due: " +
total
]
```

DS-01 is the central business rule. It first compares the dates, then fetches the selected room with Room[ID == input.Room] and checks its status, and finally counts the active allocations of the room (those whose check-out date is empty) with a criteria-based count() and compares the count with the room's capacity. Each failed check shows an alert and cancels the submission. DS-02 is a one-condition validation. DS-03 repeats the active-allocation count after the record is saved and updates the room status. DS-04 and DS-05 use sendmail with zoho.adminuserid as both sender and recipient, so mail goes only to the application administrator.

Result observed: DS-01, DS-02 and DS-03 behaved as designed in the tests of Chapter 6. DS-04 saved the complaint, but delivery of its email to the administrator inbox was not confirmed. DS-05 is scheduled to start on 2 November 2026; its Execute test was clicked but the result was not seen, so its output is not confirmed.

5.6 Workflows and Automation

Table 5.3 lists the six workflows. Four are form workflows, one is a schedule workflow and one is an approval process. The approval process was exercised: the three test residents were submitted for approval, and the administrator's Pending Tasks list was cleared afterwards. Blueprint and batch workflows were not used because there is no multi-stage lifecycle or bulk operation in this application; the one-step approval was enough for resident registration.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Validate Allocation	Form Workflow	On Validate (Add)	Always	Alert and cancel submit on invalid dates, status or capacity	DS-01
Validate Payment	Form Workflow	On Validate (Add)	Always	Alert and cancel submit if Amount_Paid <= 0	DS-02
Update Room Status	Form Workflow	On Success (Add)	Active allocations >= capacity	Set Room.Status to Occupied	DS-03
Complaint Email Alert	Form Workflow	On Success (Add)	Always	Send email to the administrator	DS-04
Weekly Rent Due Reminder	Schedule	Weekly from 02-Nov-2026 09:00	Payment Status = Pending	Send reminder email to the administrator	DS-05
Resident Registration Approval	Approval	On record add (Resident)	Always	Level-1 approver: application administrator; approve or reject	—

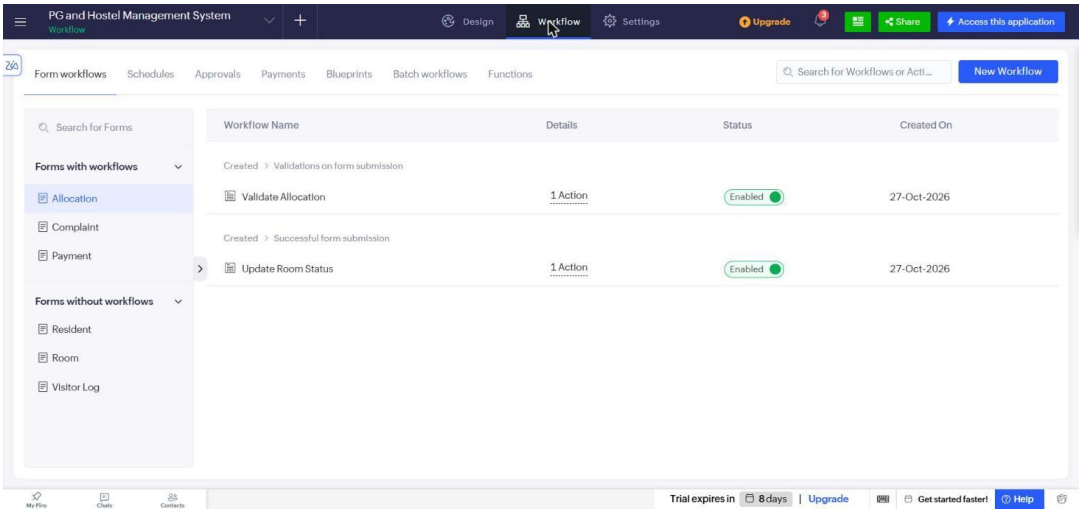


Figure 5.6: Workflow Configuration

5.7 Integrations

The only integration is the built-in email action, implemented with Deluge sendmail in DS-04 and DS-05 (see Code 5.4 and Code 5.5). Because sender and recipient are both zoho.adminuserid, no email address is hard-coded and no credential is stored. No invokeurl call, connection, webhook or Creator REST API was used, so no Postman test was needed. Delivery of the emails to the administrator's inbox could not be confirmed from the application side.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email notification	Zoho Creator built-in email	sendmail task in Deluge	None needed (sent from zoho.adminuserid within the application)	Complaint details (DS-04); count and total of pending payments (DS-05); sent only to the administrator
REST API call	Not used	invokeurl / connection	Not applicable	No external service is needed for this application
Webhook / Creator REST API	Not used	Not applicable	Not applicable	Not applicable

5.8 User Management and Role-Based Access

Access control is implemented with permission profiles and a role hierarchy (Table 4.8). Two profiles were created: Warden, with form-level Access to all six forms and to the Dashboard, and Resident, with Access to the Complaint form and the Dashboard only. Three roles form the hierarchy: CEO (the default top role, used as Administrator), Warden and Resident, in that order. Creator's View, Edit and Delete options open per-report pickers, so those finer permissions were not set. No users were invited and no record-level criteria or customer portal were configured, so role behaviour was not tested by logging in as a Warden or a Resident.

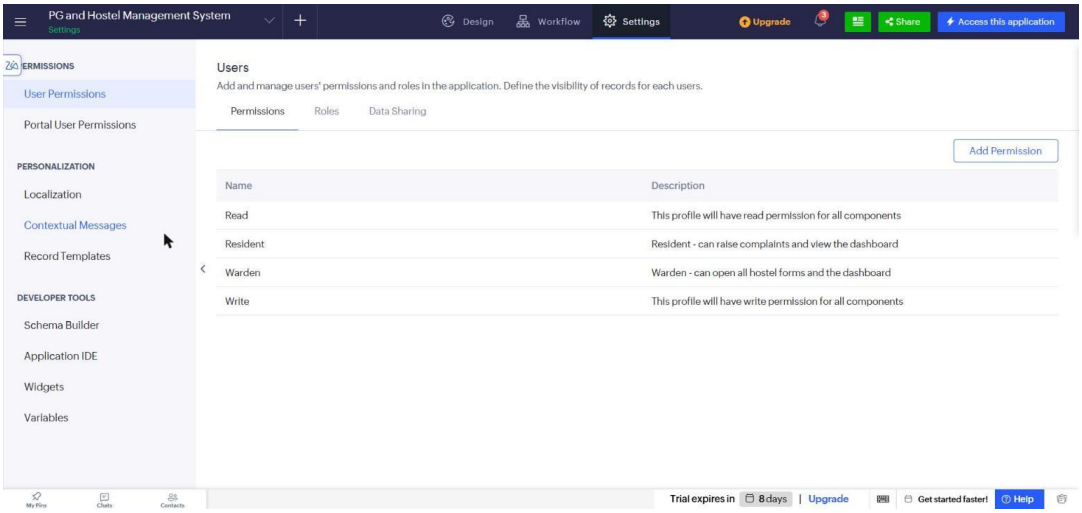


Figure 5.7: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

Zia and the other Zoho AI features (AI field, prediction, OCR, sentiment analysis, AI application builder) were not used. Analytics is provided by the three charts on the Dashboard page (Figure 5.5): Occupancy by Room Type, Payments by Status and Complaints by Status. They were configured manually and checked against the test data.

5.10 Pro-Code Extension

No pro-code extension was used: there are no Zoho Catalyst services, custom functions, JavaScript widgets or Creator REST API calls, because every requirement was met by forms, workflows and short Deluge scripts. If the application grows, a custom function could hold the allocation check so that several forms can reuse it, a Catalyst function could generate PDF receipts, and the Creator REST API could let an external attendance or payment system add records.

5.11 Mobile and Portal Access

Mobile and portal access were not used in this project. The application was used only in a web browser. The Zoho Creator mobile app could be used by the warden for rent entry on the move, and a customer portal could let residents raise complaints without a Zoho account; both are listed under future enhancements.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done in the live Development environment of the application using clearly fake data only (for example residents named Test Alpha, Test Beta and Test Gamma with emails at example.com, rooms 101, 102 and 103). Unit testing exercised each form and each Deluge validation separately, integration testing exercised the workflow chain (allocation → validation → room status update) and the reports and charts built on the same records, and the approval process was run on the test residents. Role-based testing by logging in as a Warden or a Resident, and user acceptance testing with other people, were not carried out because no users were invited.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Register a resident with all mandatory fields	Name Test Alpha, email at example.com, phone entered	Record created	Record created (three test residents added)	Pass
TC-02	FR-01	Register a resident with an existing email	Reuse the email of Test Alpha	Submission blocked	Blocked: “This value already exists. Enter a unique value.”	Pass
TC-03	FR-02	Approve the registered residents	Open Pending Tasks and approve	Pending Tasks list cleared	Pending list empty; Alpha and Beta shown as completed, Gamma not listed under completed	Partial
TC-04	FR-04	Allocate Test Alpha to room 101 (capacity 1)	Resident, Room 101, check-in 06-Nov-2026	Allocation saved	Saved	Pass
TC-05	FR-06	Allocate a resident to room 103 (Under Maintenance)	Room 103	Submission blocked	Blocked: “Room status is: Under Maintenance”	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-06	FR-06, FR-07	Allocate again to room 101 after it became full	Room 101	Blocked; room now Occupied	Blocked: “Room status is: Occupied”	Pass
TC-07	FR-05	Check-out earlier than check-in	Check-in 06-Nov, check-out 01-Nov	Submission blocked	Blocked: “Check-out date cannot be earlier than the check-in date.”	Pass
TC-08	FR-04, FR-07	Allocate Beta and Gamma to room 102 (capacity 2)	Two allocations in room 102	Both saved; room becomes Occupied	Both saved; Room Occupancy report shows 102 Occupied	Pass
TC-09	FR-08	Payment with amount 0	Amount_Paid = 0	Submission blocked	Blocked: “Amount paid must be greater than 0.”	Pass
TC-10	FR-08, FR-09	Payment of 4000 against a room with rent 6000	Amount_Paid = 4000, Status Pending	Saved; Balance Due = 2000	Saved; Balance Due showed 2000.00	Pass
TC-11	FR-13	Open Pending Payments report	Report with Status = Pending filter	Pending record listed	Listed Nov-2026, UPI, Pending, 4000.00, Test Alpha, room 101; Balance Due column not shown	Partial
TC-12	FR-13	Open Room Occupancy report	Room report	Rooms with status	101 Occupied, 102 Occupied, 103 Under Maintenance	Pass
TC-13	FR-14	Open the Dashboard	Dashboard page	Three charts render with data	Room-type and pending-payment charts and complaint chart rendered; third chart’s bars not scrolled into view	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-14	FR-10	Submit an Electrical complaint	Category Electrical, Status Open	Saved; email to administrator	Complaint saved; email delivery not confirmed	Partial
TC-15	FR-11	Run the weekly reminder	Execute the schedule script	Email with count and total	Execute clicked; result not seen	Not confirmed
TC-16	FR-12	Add a visitor log entry	Test Visitor One, 27-Oct-2026 17:00–18:00	Saved and listed in the Visitor Log Report	Saved and shown in the report	Pass
TC-17	FR-15, NFR-02	Open the Dashboard as a Warden or a Resident	Log in as another role	Only permitted items visible	Not tested (no users invited)	Not tested

6.3 Debugging Deluge Scripts

Scripts were debugged by running them through the live forms and reading the result. The alert text shown on a rejected submission shows exactly which rule fired (for example the room status in DS-01), so each alert message was written to name the condition. The Deluge editor was used to save and check syntax, and the Execute option was used on the scheduled script (its result was not seen). No Deluge runtime error was observed in the final scripts. One design point was found while writing DS-01: the room must be counted using only allocations with an empty check-out date, otherwise a resident who has already left would still occupy a bed; the criteria Check_Out_Date == null in DS-01 and DS-03 fixes this.

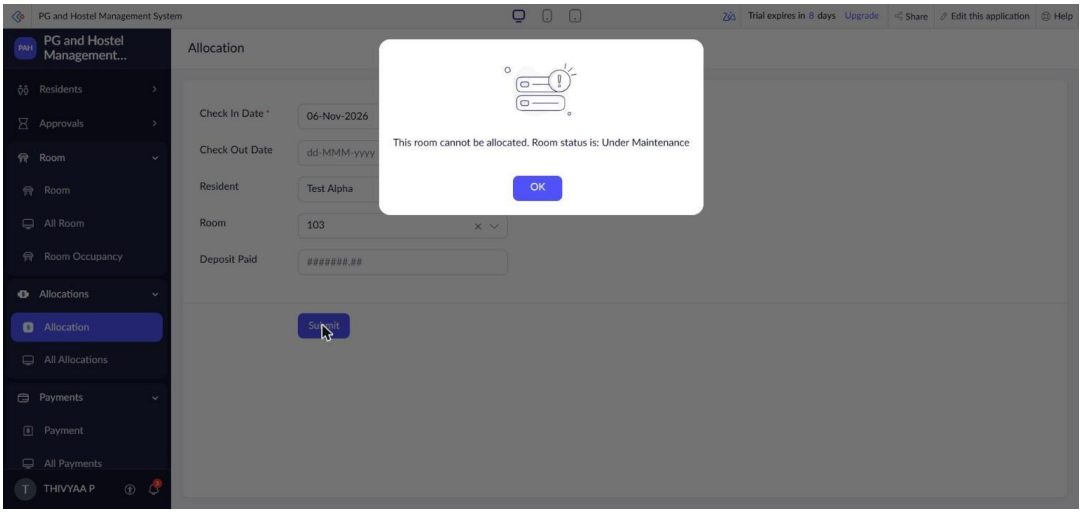


Figure 6.1: Validation Alert Shown on Submission of the Allocation Form

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Link names were doubled while building fields (for example CapacityCapacity, StatusStatus, Room_TypeRoom_Type)	Typing appended to the existing text of the field-name box instead of replacing it	Selected all text in the box and retyped the correct link name; verified every link name	Closed
BUG-02	The text “Allocation” was typed into the Capacity field name	Keyboard focus was on the wrong box	Restored the field name to Capacity	Closed
BUG-03	Permission profile description was rejected	Creator does not accept a semicolon in the description	Rewrote the description using a hyphen	Closed
BUG-04	Balance Due column cannot be shown in the Pending Payments report	The formula field is not offered by the report column chooser	Not fixed; the value is still visible on the Payment form	Open
BUG-05	Report designer pages timed out repeatedly while editing	Intermittent slow response of the designer	Retried; the BUG-04 change could not be completed	Open

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Self-test by the developer	Administrator (owner account)	Full scenario run with fake data: all validations behaved as designed; two report/permission limits noted	Defects BUG-04 and BUG-05 logged in Table 6.2
No other users	Warden, Resident	Not available: no users were invited, so no external feedback was collected	To be done before any real use

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was built and tested only in the Development environment. Stage and Production environments were not used and the application was not published to Production, so there is no version note or rollback plan beyond the ability to edit the application again in the builder. The Free Trial account was used, and the trial ends about eight days after the build, so a paid subscription or an Education plan would be needed to keep the application running.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓ (in Development; TC-03, TC-11, TC-14, TC-15, TC-17 partial or pending)
2	Sample/test data removed or isolated	✗ (fake test records remain)
3	Roles, permissions and users configured in Production	✓ (roles and profiles; no users invited)
4	Connections authorised in Production	N/A (no connections)
5	Schedules and workflows enabled	✓ (4 form workflows, schedule and approval enabled)
6	Application published and URL verified	✗ (not published to Production)
7	Version / release notes recorded	✗

7.2 Performance Optimisation

Performance was handled by design rather than measured. Allocation checks use criteria-based count() instead of fetching all allocations and looping; the weekly

reminder fetches the pending payments once and loops over that list with no fetch inside the loop; and the Pending Payments report is filtered by status. No timing measurements were taken, so no before and after figures are claimed.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria-based count() instead of fetching and looping	DS-01, DS-03	Not measured	Not measured	Design choice only
Single fetch, no fetch inside the loop	DS-05	Not measured	Not measured	Design choice only
Filtered report (Status = Pending)	Pending Payments report	Not measured	Not measured	Design choice only

7.3 Security Features

Security measures configured are limited to what is described in Table 7.3. Encrypted fields, IP or domain restrictions, two-factor authentication and OAuth connections were not configured, because no external connection or sensitive payment data is used. Only clearly fake test data was entered, and no payment card or password details were entered anywhere in the application.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Permission profiles (Warden, Resident)	Form-level Access to forms and the Dashboard	Unauthorised access to forms
Mandatory and unique field rules	Resident.Email, Room.Room_Number, mandatory amounts and dates	Duplicate and incomplete records
Server-side Deluge validation	Allocation and Payment On Validate	Invalid or inconsistent data
Emails only to zoho.adminuserid	DS-04, DS-05	Leakage of data to unintended recipients; hard-coded addresses
No stored keys or passwords	Whole application	Credential leakage

7.4 Governance and Administration

Governance was kept simple: one owner account administers the application; forms, fields and workflows use consistent names (Title_Case link names, workflow names

that state their purpose); and the permission profiles and role hierarchy separate the Warden and Resident from the Administrator. The audit trail, change history, data export and backup, and licence management settings were not reviewed or configured in this project. Maintenance is the responsibility of the owner account; the Warden and Resident profiles do not have designer access.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario below follows one typical day in the PG. First, the warden tries to allocate a resident to room 103, which is Under Maintenance; the application shows an alert and cancels the submission (Figure 8.1). The warden then allocates Test Alpha to room 101, whose capacity is one; the allocation is saved and room 101 becomes Occupied (Figure 8.2). Finally a payment of 4000 is recorded against the 6000 rent; the Balance Due formula shows 2000, the record appears in the Pending Payments report and is counted in the Payments by Status chart on the Dashboard (Figure 8.3).

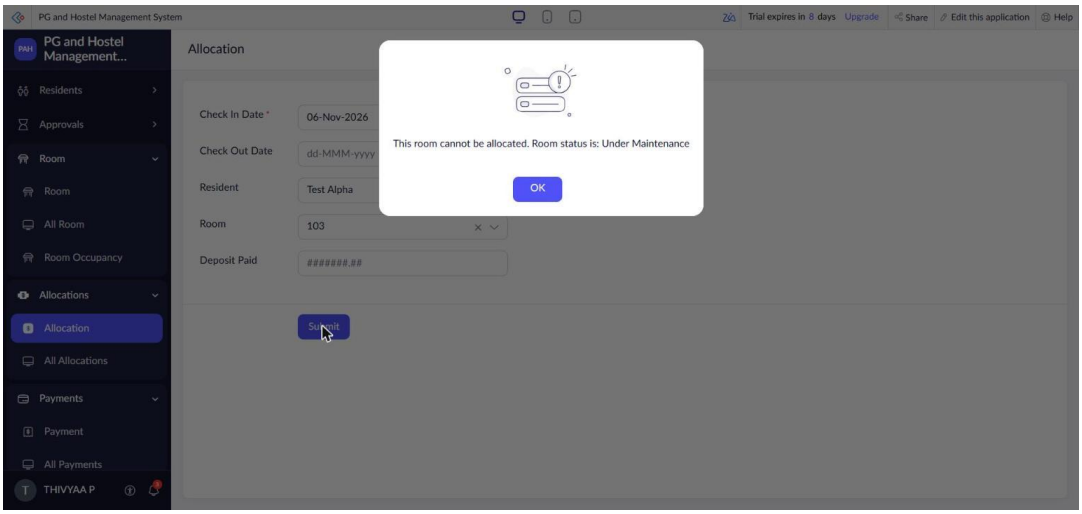


Figure 8.1: Scenario Step 1 – Allocation to a Room Under Maintenance Is Blocked

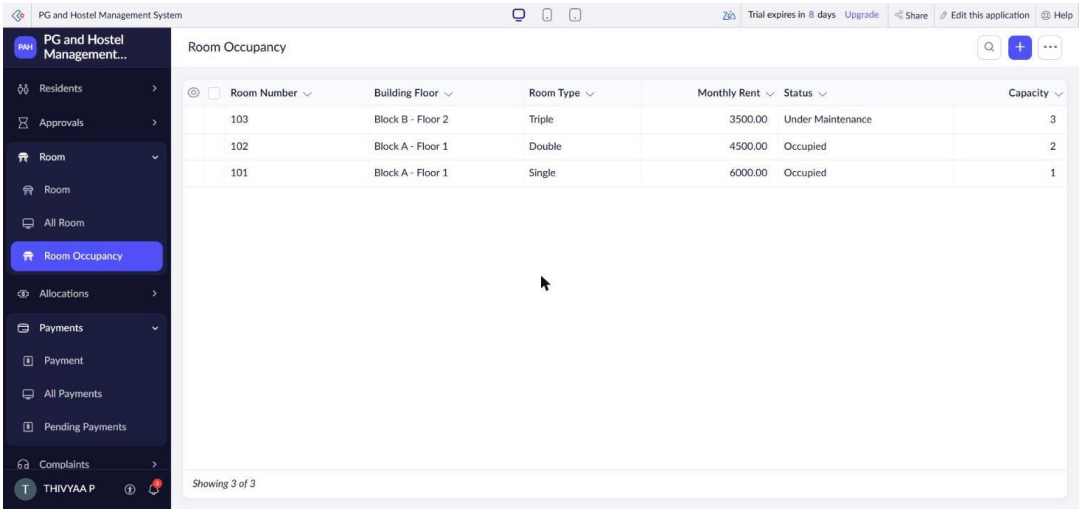


Figure 8.2: Scenario Step 2 – Allocation Saved and Room Becomes Occupied

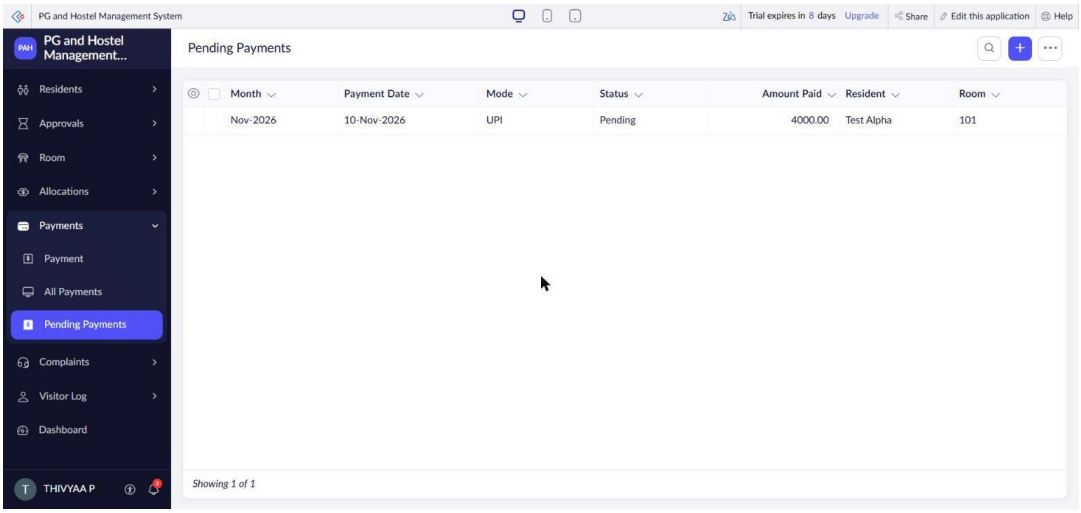


Figure 8.3: Scenario Step 3 – Payment with Balance Due and Pending Payments Report

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
To design a normalised data model and forms	Sections 4.2, 4.3, 5.2	Six forms with lookups and a formula field (Tables 4.1–4.7); achieved
To automate validation, room status, alerts and reminders	Sections 5.5, 5.6	Five Deluge scripts and six workflows; validations tested (TC-02 to TC-10); email delivery and schedule run not confirmed; mostly achieved
To provide role-based dashboards and reports	Sections 5.3, 5.4, 5.8	Eight list reports, one dashboard, two profiles; Balance Due missing in one report and role behaviour untested; partly achieved
To use built-in email integration	Section 5.7	sendmail in DS-04 and DS-05; delivery not confirmed; partly achieved
To record deployment, security and governance honestly	Chapter 7	Documented as Development only; Stage, Production, audit trail and backup not done; achieved as documentation

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Several minutes to find and write in registers	One form submission with instant validation	Faster and checked at entry
Manual steps	Copy details to registers, check room by memory, calculate dues by hand	Select lookups; the system checks, calculates the balance and updates room status	Fewer manual steps and fewer errors
Report generation	Monthly summary prepared by hand	List reports and dashboard charts update from the data	Always current
Data accessibility	Paper registers kept with the warden	Single web application for all roles	One place, access by role

8.4 Limitations of the Low-Code Approach

Constraints met on Zoho Creator and how they were handled: (1) The Balance Due formula field is not offered by the Pending Payments report column chooser, so the due amount is visible on the Payment form but not in that report; this was left open. (2) Creator's View, Edit and Delete permission options open per-report pickers, so profile permissions were limited to form-level Access. (3) Some builder pages were slow and timed out repeatedly, which delayed report editing. (4) The free trial is time-limited, and a scheduled workflow cannot be seen running until its first scheduled date. (5) The Deluge fetch-and-count approach is simple and clear for a small PG, but a large hostel would need to watch execution limits. (6) Vendor lock-in: the application depends on Zoho Creator's hosting and data model.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the no-code to pro-code spectrum, most of this application is no-code configuration: forms and fields, lookups, the formula field, list reports, the dashboard charts, permission profiles and the approval process. Low-code Deluge was needed only for the five scripts DS-01 to DS-05, which hold the rules that the visual builder cannot express (counting active allocations, comparing capacity, updating another form and building an email). No pro-code was needed. I would move to pro-code, for example

with Catalyst or the Creator REST API, when the hostel needs payment gateway integration, bulk data imports, or an external attendance or door system.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The PG and Hostel Management System was built on Zoho Creator with six linked forms, a formula field, five Deluge scripts, six workflows, eight list reports, a three-chart dashboard and Warden and Resident permission profiles. Testing with fake data showed that duplicate emails, unavailable or full rooms, reversed dates and zero payments are rejected, that rooms become Occupied automatically, and that Balance Due and the reports and charts are correct.

Some items remain incomplete or unconfirmed: delivery of the complaint and weekly reminder emails, the first run of the weekly schedule, the Balance Due column in the Pending Payments report, fine-grained View and Edit permissions, role-based testing by real users, and deployment beyond the Development environment. These are listed so that the report matches the live application.

9.2 Future Enhancements

- Show Balance Due in the Pending Payments report and set record-level View and Edit permissions for the Warden and Resident.
- Add a room-change request with its own approval process, and a Maintenance Request form linked to complaints.
- Invite real users and run role-based and user acceptance testing, then publish through Stage to Production.
- Add a resident self-service portal and use the Zoho Creator mobile app for rent entry.
- Integrate an online payment gateway and generate PDF receipts using a Catalyst function or a custom function.
- Use Zia or Zoho Analytics for occupancy forecasting and defaulter prediction.

9.3 Learning Outcomes and Career Relevance

Building this application gave practical experience of data modelling with lookups, field and form design, Deluge scripting for validation and notification, form, schedule and approval workflows, reports and pages, and role-based access. It also showed the limits of the platform and the importance of testing and of recording honestly what was

not completed. These skills map to the roles of citizen developer, low-code developer, solutions consultant and automation engineer, and they provide a base for later learning of Catalyst and the Creator REST API.

REFERENCES

[1] A. C. Bock and U. Frank, "Low-code platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 27 October 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 27 October 2026).

[4] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. 46th Euromicro Conf. on Software Engineering and Advanced Applications (SEAA)*, 2020.

[5] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, "Characteristics and challenges of low-code development: The practitioners' perspective," in *Proc. 15th ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM)*, 2021.

[6] M. Waszkowski, "Low-code platform for automating business processes in manufacturing," *IFAC-PapersOnLine*, vol. 52, no. 10, pp. 376–381, 2019.

[7] J. Cabot, "Positioning of the low-code movement within the field of model-driven engineering," in *Proc. ACM/IEEE 23rd Int. Conf. on Model Driven Engineering Languages and Systems: Companion Proceedings (MODELS-C)*, 2020.

[8] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application "PG and Hostel Management System"; sections Forms, Reports, Dashboard	5.1	Fig 5.1
II	Forms with ≥ 8 distinct field types	Y	Resident, Room, Allocation, Payment, Complaint, Visitor Log (12 field types)	5.2	Tables 4.2–4.7
II	Lookup field (relationship between forms)	Y	Allocation, Payment, Complaint, Visitor Log lookups	4.2, 5.2	Fig 4.2; Table 4.1
II	Subform (parent–child data)	N	Not used; child records are separate forms	4.2, 5.2	—
II	Field validations / layout rules	Partial	Mandatory and unique fields; Deluge validations; layout rules not used	5.2	Code 5.1, 5.2
II	List report with filter, grouping, custom action	Partial	Pending Payments (filter, sort); no grouping or custom action	5.3	Fig 5.4; Table 5.1
II	Kanban / Calendar / Timeline / Map report	N	Not built	5.3	—
II	Pivot table / chart report	N	Charts are on the Dashboard page, not chart reports	5.3	—
II	Page / dashboard built with UI Builder	Y	Dashboard page with 3 charts and an embedded report	5.4	Fig 5.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – form events (validate / success / field actions)	Y	Allocation (validate, success), Payment (validate), Complaint (success)	5.5	Code 5.1–5.4
II	Deluge – functions, fetch, loops, collections	Partial	fetch, count(), for each, ifnull; no custom function	5.5	Code 5.1, 5.5
II	Email / in-app notifications	Y	sendmail in Complaint Email Alert and Weekly Rent Due Reminder (delivery not confirmed)	5.5, 5.6	Code 5.4, 5.5
II	Form workflow	Y	Four form workflows	5.6	Table 5.3; Fig 5.6
II	Scheduled workflow	Y	Weekly Rent Due Reminder (first run 02-Nov-2026; result not confirmed)	5.6	Code 5.5
II	Approval process	Y	Resident Registration Approval	5.6	Table 5.3
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	Test cases, defect log, validation alerts	6	Tables 6.1, 6.2; Fig 6.1
III	Integration with a Zoho service	Partial	Built-in email (sendmail) only	5.7	Table 5.4
III	Connection / REST API call (invokeurl)	N	Not used	5.7	—
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Partial	Warden and Resident profiles; CEO, Warden, Resident roles; no users invited	5.8	Table 4.8; Fig 5.7

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Record-level access criteria / customer portal	N	Not configured	5.8	—
III	Zia / AI feature	N	Zia / AI not used by design	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Three charts on the Dashboard page	5.9	Fig 5.5
IV	Performance optimisation applied & measured	N	Applied by design, not measured	7.2	Table 7.2
IV	Deployment across environments	N	Development environment only	7.1	—
IV	Security controls (encryption, OAuth, permissions)	Partial	Role permissions, unique and mandatory rules, validations; no encryption or OAuth	7.3	Table 7.3
IV	Governance (audit trail, administration)	N	Audit trail not reviewed	7.4	—
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum analysis	8.4, 8.5	Sections 8.4, 8.5
—	Mobile app / portal access	N	Not used	5.11	—

Total features implemented: 12 / 32 (Fully: 12, Partially: 6, Not implemented: 14)

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	1	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Validate Allocation

```
// Script ID : DS-01
// Location  : Allocation → On Validate (Add)
// Purpose    : Block the submission if the dates, room status or
capacity are invalid

if(input.Check_Out_Date != null && input.Check_Out_Date <
input.Check_In_Date)
{
    alert "Check-out date cannot be earlier than the check-in date.";
    cancel submit;
}
roomRec = Room[ID == input.Room];
if(roomRec.Status != "Available")
{
    alert "This room cannot be allocated. Room status is: " +
roomRec.Status;
    cancel submit;
}
currentCount = Allocation[Room == input.Room && Check_Out_Date ==
null].count();
if(currentCount >= roomRec.Capacity)
{
    alert "This room is already full. Capacity: " + roomRec.Capacity;
    cancel submit;
}
```

C.2 Validate Payment

```
// Script ID : DS-02
// Location  : Payment → On Validate (Add)
// Purpose    : Block a payment whose amount is 0 or less

if(input.Amount_Paid <= 0)
{
    alert "Amount paid must be greater than 0.";
    cancel submit;
}
```

C.3 Update Room Status

```
// Script ID : DS-03
// Location  : Allocation → On Success (Add)
// Purpose    : Set the room to Occupied when it has reached capacity

roomRec = Room[ID == input.Room];
activeCount = Allocation[Room == input.Room && Check_Out_Date ==
null].count();
if(activeCount >= roomRec.Capacity)
{
    roomRec.Status = "Occupied";
}
```

C.4 Complaint Email Alert

```
// Script ID : DS-04
// Location  : Complaint → On Success (Add)
// Purpose    : Notify the application administrator of a new
complaint

sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"New hostel complaint: " + input.Category
    message :"A new complaint was registered. Category: " +
input.Category + ". Description: " + input.Description + ". Status: "
+ input.Status
]
```

C.5 Weekly Rent Due Reminder

```
// Script ID : DS-05
// Location  : Schedule workflow: weekly from 02-Nov-2026 09:00
// Purpose    : Email the administrator the count and total balance of
pending payments

pendingList = Payment[Status == "Pending"];
cnt = pendingList.count();
total = 0;
for each p in pendingList
{
    total = total + ifnull(p.Balance_Due, 0);
}
sendmail
[
    from :zoho.adminuserid
    to :zoho.adminuserid
    subject :"Weekly rent due reminder"
    message :"Pending payments: " + cnt + ". Total balance due: " +
total
]
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application link from your Zoho Creator account (creatorapp.zoho.in) in a web browser and sign in with your Zoho account. The Forms, Reports and Dashboard sections appear according to your role. The mobile app and customer portal were not configured.

D.2 Instructions for Administrator

- Sign in as the application owner (Administrator).
- Open Dashboard to see rooms by type, payments by status, complaints by status and the Pending Payments report.
- Open Pending Tasks to approve or reject new resident registrations.
- Open Room Occupancy to check the status of each room.
- Check the mailbox of the owner account for the “New hostel complaint” and “Weekly rent due reminder” emails.
- Use Settings to review roles and permission profiles.

D.3 Instructions for Warden

- Open the Resident form and add a new resident with name, unique email and phone.
- Open the Room form to add or update rooms (number, type, capacity, rent, status).
- Open the Allocation form, pick the resident and room, enter the check-in date and submit; read any alert and correct the entry.
- Open the Payment form, pick the resident and room, enter the month, mode, status and amount paid (greater than 0); Balance Due is calculated automatically.
- Open the Visitor Log form to record a visitor’s name and in and out times.
- Open All Payments, Pending Payments and Room Occupancy for the current position.

D.4 Instructions for Resident

- Open the Complaint form, pick your name, choose the category, describe the problem and submit.
- Open the Dashboard to view the occupancy, payment and complaint charts.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Thivyaa P	Register No.	240701570
Project Title	PG and Hostel Management System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Thivyaa P	Register No.	240701570
Department / Section	Computer Science and Engineering	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____